

Rehospitalization of Diabetes Patients: EDA

Overview

This notebook explores the UC Irvine *Diabetes 130-US Hospitals for Years 1999-2008* dataset to understand how each feature relates to our target variable, rehospitalization. Each row of the dataset references a diabetic patient's hospitalization. Features describe the patient's race, age, gender, time in the hospital, number of lab procedures, medications, and diagnoses. 22 columns correspond to diabetes medications, like insulin and Metformin; these features describe whether or not the patient is prescribed that medication, and, if so, whether their dosage was kept steady, raised, or lowered.

I will begin by loading the libraries used for this analysis. Pandas and NumPy are used to load the data and make calculations with its features. Plotly is used to visualize the data.

```
In [2]: import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
```

Next, I'll load the dataset from a local directory and display the first few rows.

```
In [4]: path = "../data/df_diabetic.csv"
df = pd.read_csv(path, low_memory=False)

with pd.option_context('display.max_columns', None):
    display(df.head())
```

	encounter_id	patient_nbr	race	gender	age	weight	admission_type_id	disch
0	2278392	8222157	Caucasian	Female	[0-10)	NaN		6
1	149190	55629189	Caucasian	Female	[10-20)	NaN		1
2	64410	86047875	AfricanAmerican	Female	[20-30)	NaN		1
3	500364	82442376	Caucasian	Male	[30-40)	NaN		1
4	16680	42519267	Caucasian	Male	[40-50)	NaN		1

The diagnosis columns - diag_1, diag_2, and diag_3 - are all floats and refer to the ICD9 encoding for each condition. I'll write two helper functions to match these codes to specific

labels and diagnosis groupings, using the labels defined in this paper listed on the dataset repository page: <https://onlinelibrary.wiley.com/doi/10.1155/2014/781670>

```
In [6]: def match_icd9(diag_col):
    #ICD9 codes referring to an external cause begin with a letter E-V. I'll replace
    diag_col = diag_col.str.replace(r'[E-V]\d*', '1000', regex = True).astype(float)

    #I'll use np.select to parallelize matching numeric values with string labels.
    #These are the numeric bounds corresponding to each label
    conditions = [
        diag_col.between(1, 139), #infectious and parasitic diseases
        diag_col.between(140, 239), #neoplasms (cancer/other tissue growths)
        (diag_col.between(240, 279)) & (np.floor(diag_col) != 250), #endocrine, nutritional,
        np.floor(diag_col) == 250, #diabetes mellitus
        diag_col.between(280, 289), #diseases of the blood and blood-forming organs
        diag_col.between(290, 319), #mental disorders
        diag_col.between(320, 359), #diseases of the nervous system
        diag_col.between(360, 389), #diseases of the sense organs
        (diag_col.between(390, 459)) | (diag_col == 785), #diseases of the circulatory system
        (diag_col.between(460, 519)) | (diag_col == 786), #diseases of the respiratory system
        (diag_col.between(520, 579)) | (diag_col == 787), #diseases of the digestive system
        (diag_col.between(580, 629)) | (diag_col == 788), #diseases of the genitourinary system
        diag_col.between(630, 679), #complications of pregnancy, childbirth, and the puerperium
        (diag_col.between(680, 709)) | (diag_col == 782), #diseases of the skin and subcutaneous tissue
        diag_col.between(710, 739), #diseases of the musculoskeletal system and connective tissue
        diag_col.between(740, 759), #congenital anomalies
        diag_col.between(800, 999), #injury and poisoning
        diag_col == 1000, #external causes of injury and supplemental classification
        diag_col.isna() #na
    ]

    #These labels correspond to the ICD9 description for each condition
    labels = [
        'infectious and parasitic diseases',
        'neoplasms',
        'endocrine, nutritional, and metabolic diseases and immunity disorders',
        'diabetes mellitus',
        'diseases of the blood and blood-forming organs',
        'mental disorders',
        'diseases of the nervous system',
        'diseases of the sense organs',
        'diseases of the circulatory system',
        'diseases of the respiratory system',
        'diseases of the digestive system',
        'diseases of the genitourinary system',
        'complications of pregnancy, childbirth, and the puerperium',
        'diseases of the skin and subcutaneous tissue',
        'diseases of the musculoskeletal system and connective tissue',
        'congenital anomalies',
        'injury and poisoning',
        'external causes of injury and supplemental classification',
        'na'
    ]

    return pd.Series(np.select(conditions, labels, default = 'other'))
```

```
#Tests the function on the primary diagnosis for each patient
match_icd9(df['diag_1']).value_counts(normalize = True)
```

```
Out[6]: diseases of the circulatory system      0.299088
diseases of the respiratory system      0.141727
diseases of the digestive system      0.093106
diabetes mellitus      0.086050
injury and poisoning      0.068530
diseases of the genitourinary system      0.050282
diseases of the musculoskeletal system and connective tissue      0.048710
neoplasms      0.033734
other      0.030747
infectious and parasitic diseases      0.027200
endocrine, nutritional, and metabolic diseases and immunity disorders      0.026551
diseases of the skin and subcutaneous tissue      0.025686
mental disorders      0.022227
external causes of injury and supplemental classification      0.016165
diseases of the blood and blood-forming organs      0.010839
diseases of the nervous system      0.009306
complications of pregnancy, childbirth, and the puerperium      0.006751
diseases of the sense organs      0.002594
congenital anomalies      0.000501
na      0.000206
Name: proportion, dtype: float64
```

Not all patients have a secondary/tertiary diagnosis, so I'll examine diag_1 only, for now. The top 4 causes for hospitalization are circulatory diseases, respiratory diseases, digestive system diseases, and diabetes-related complications. Most of these labels correspond to a small proportion of patients, so I'll group more of these into the 'Other' category.

```
In [8]: def match_icd9_group(diag_col):
#ICD9 codes referring to an external cause begin with a letter E-V. I'll replace
diag_col = diag_col.str.replace(r'[E-V]\d*', '1000', regex = True).astype(float)

#Matches each condition with its ICD9 grouping
conditions = [
    (diag_col.between(390, 459)) | (diag_col == 785), #circulatory
    (diag_col.between(460, 519)) | (diag_col == 786), #respiratory
    (diag_col.between(520, 579)) | (diag_col == 787), #digestive
    np.floor(diag_col) == 250, #diabetes
    diag_col.between(800, 999), #injury
    diag_col.between(710, 739), #musculoskeletal
    (diag_col.between(580, 629)) | (diag_col == 788), #genitourinary
    diag_col.between(140, 239), #neoplasms (cancer/other tissue growths)
    diag_col.isna() #na
] #anything else is defined as 'other'

groups = [
    'circulatory',
    'respiratory',
    'digestive',
    'diabetes',
    'injury',
    'musculoskeletal',
```

```

        'genitourinary',
        'neoplasms',
        'na'
    ]

    return pd.Series(np.select(conditions, groups, default = 'other'))

df['diag_1g'] = match_icd9_group(df['diag_1'])
df['diag_1g'].value_counts(normalize = True)

```

```

Out[8]: diag_1g
circulatory      0.299088
other            0.178567
respiratory      0.141727
digestive        0.093106
diabetes         0.086050
injury           0.068530
genitourinary    0.050282
musculoskeletal  0.048710
neoplasms        0.033734
na               0.000206
Name: proportion, dtype: float64

```

As found before, most patients are admitted for circulatory, respiratory, digestive, or diabetes issues. Diagnosis data is missing for very few patients, so I will omit these rows from the EDA.

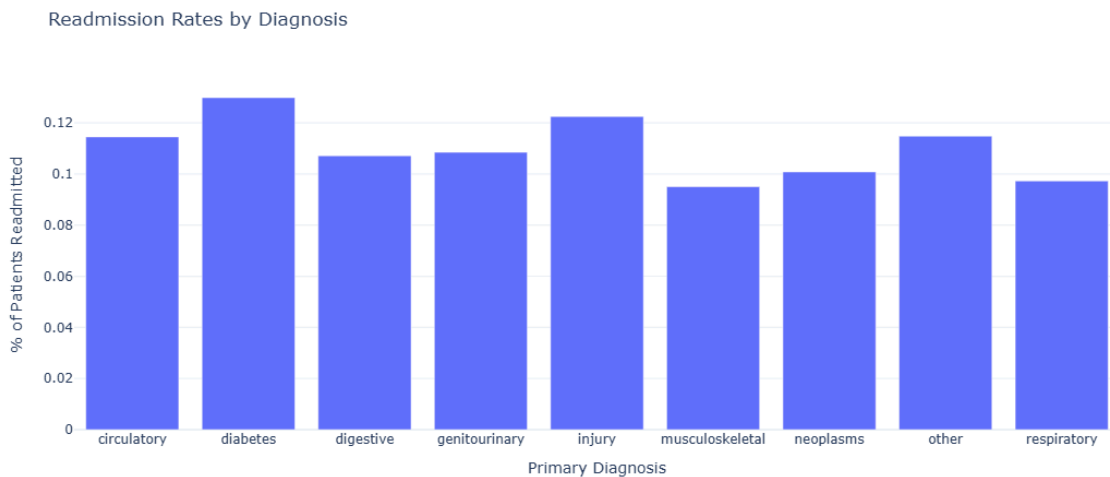
```

In [10]: #Omits all rows where primary diagnosis is missing
df = df[df['diag_1g'] != 'na']

#Calculates the proportion of patients readmitted for each diagnosis group
diag_readmit = df.groupby('diag_1g')['readmitted'].mean().to_frame('pct_readmitted')

#Plots a bar chart showing the % readmitted by diagnosis
fig = px.bar(diag_readmit, y = 'pct_readmitted', template = 'plotly_white', title = 
             height = 500, width = 800)
fig.update_layout(xaxis_title = 'Primary Diagnosis', yaxis_title = '% of Patients R
fig.show()

```



```
In [11]: fig = px.bar((df.groupby('diag_1g')['diag_1'].nunique()/len(df))*100, template = 'plotly',
                  title = 'Proportion of Unique ICD-9 Codes per Diagnosis Group')
fig.update_layout(xaxis_title = 'Diagnosis Group', yaxis_title = '% of Unique Codes')
fig.update_traces(textposition='outside', texttemplate='%{y:.2f}%')
fig.show()
```

Readmission rates are similar across all diagnosis groups. It appears that diabetes and injury have higher rates of readmission, while musculoskeletal and respiratory have lower rates. We could investigate further to determine if these differences are statistically significant. I'll next plot a boxplot to visualize the distribution of days spent in the hospital by patients in each diagnosis group. I'll use the color parameter to understand how time spent in the hospital affects readmission.

```
In [13]: fig = px.box(df, x = 'diag_1g', color = 'readmitted', y = 'time_in_hospital', template = 'plotly',
                    fig.update_layout(xaxis_title = 'Diagnosis', yaxis_title = 'Time In Hospital (Days)')
fig.show()
```

The median time_in_hospital for readmitted patients with diagnosis diabetes, circulatory, respiratory, or musculoskeletal is 1 day longer than that of patients who were not readmitted within 30 days. Readmitted patients whose primary diagnosis was genitourinary also tend to spend more time in the hospital than non-readmitted patients.

I'll use three pivot tables to understand how race, age, and gender affect readmission rates. These pivot tables will plot the categorical columns on the x and y axes, then visualize the proportion of patients in that group who were readmitted within 30 days. It will be important to consider sample sizes when evaluating these visuals. For example, if only eight patients in the dataset are men in the age range [20-30), and four of these patients were readmitted, the group will have a 50% readmission rate. In a dataset with over 100,000 patients, this group would be too small to be statistically significant.

```
In [16]: #Calculates and visualizes the % of patients readmitted by age and race
fig = px.imshow(pd.pivot_table(df, index = 'race', columns = 'age', values = 'readmitted',
                              template = 'seaborn', title = '% of Patients Readmitted by Race and Age'))
fig.show()
```

I expected readmission risk would correlate with age, so it is surprising to see that patients in the [20-30) range have the highest readmission rate. Rates appear similar across all race groups, with Caucasians and African Americans having slightly higher readmission rates. This visual indicates that older hispanics are at higher risk for readmission, which may indicate inequalities in access to better treatment. Upon further examination, I see that 19 Hispanic patients are in the [90-100) age group, and 4 of those patients were readmitted.

```
In [18]: #Calculates and visualizes the % of patients readmitted by age and gender
fig = px.imshow(pd.pivot_table(df[df['gender'] != 'Unknown/Invalid'], index = 'age', columns = 'gender', values = 'readmitted',
                              template = 'seaborn', title = '% of Patients Readmitted by Age and Gender'))
fig.show()
```

```
template = 'seaborn', title = '% of Patients Readmitted by Gender and Age'
fig.show()
```

It appears that female diabetic patients in the [20-30) age range are at a higher risk of rehospitalization than any other group. It appears there is a nonlinear relationship between age and readmission risk, where risk decreases from age [20-30) to [50-60), then increases from age [50-60) to age [80-90). Children have a very low risk of rehospitalization compared to the other age groups.

```
In [20]: #Calculates and visualizes the % of patients readmitted by race and gender
fig = px.imshow(pd.pivot_table(df[df['gender'] != 'Unknown/Invalid'], index = 'gender', columns = 'race',
                                template = 'seaborn', title = '% of Patients Readmitted by Race and Gender'))
fig.show()
```

According to this visual, a greater proportion of Asian men were readmitted within 30 days than Asian women. Hispanic men also have higher rates than Hispanic women, while men with race Other are at lower risk than women with race Other. Rehospitalization rates among men and women are similar for African American and Caucasian patients. I expect this is due to larger sample sizes.

```
In [22]: fig = px.histogram(df, x = 'race', color = 'readmitted', template = 'plotly_white',
                             title = 'Readmission Rates by Race', height= 500, width = 800)

fig.update_layout(xaxis_title = 'Race', yaxis_title = '# of Patients')

fig.show()
```

This plot confirms that any findings related to patient race are likely due to small sample sizes. Most patients are Caucasian or African American.

22 of the columns refer to different diabetes medications. These columns have values 'No', 'Down', 'Steady', and 'Up'. To better understand how these medications influence readmission rates, I'll reshape the data so each patient's medication and its status is on its own row. This will enable me to better determine the frequency of each medication, the relationship between medications and primary diagnoses, and the impact of medication changes on readmission rates.

```
In [25]: #Selects the medication columns as all columns between the indices of 'metformin' and 'insulin'
med_cols = df.columns[df.columns.get_loc('metformin'):df.columns.get_loc('insulin')]

#Reshapes the data from wide to long, placing each patient's medication statuses on its own row
prescriptions = pd.melt(df, id_vars = ['encounter_id', 'diag_1g', 'readmitted'], value_vars = med_cols)
prescriptions.head()
```

Out[25]:

	encounter_id	diag_1g	readmitted	medication	prescribed
0	2278392	diabetes	0	metformin	No
1	149190	other	0	metformin	No
2	64410	other	0	metformin	No
3	500364	other	0	metformin	No
4	16680	neoplasms	0	metformin	No

For this analysis, it is important to understand the frequency at which each medication is prescribed. If only a small subset of patients takes a certain medication, then any findings related to that medication may not apply to the majority of patients. I'll use another pivot table to count the frequency of each medication's status.

```
In [27]: #Counts the number of patients for whom each medication is kept steady, increased,
med_counts = pd.pivot_table(prescriptions, index = 'medication', columns = 'prescri
                        aggfunc = 'count', fill_value = 0)

display(med_counts[['Steady', 'Up', 'Down', 'No']])
```

prescribed medication	Steady	Up	Down	No
acarbose	295	10	3	101437
acetoexamide	1	0	0	101744
chlorpropamide	79	6	1	101659
citoglipton	0	0	0	101745
examide	0	0	0	101745
glimepiride	4668	327	194	96556
glimepiride-pioglitazone	1	0	0	101744
glipizide	11353	770	560	89062
glipizide-metformin	13	0	0	101732
glyburide	9272	812	564	91097
glyburide-metformin	692	8	6	101039
insulin	30843	11315	12216	47371
metformin	18341	1067	575	81762
metformin-rosiglitazone	2	0	0	101743
miglitol	31	2	5	101707
nateglinide	667	24	11	101043
pioglitazone	6974	234	118	94419
repaglinide	1384	110	45	100206
rosiglitazone	6097	178	87	95383
tolazamide	38	1	0	101706
tolbutamide	23	0	0	101722
trogliptazone	3	0	0	101742

Two medications - citoglipton and examide - are not prescribed to any patients. Only insulin is prescribed more often than not, and many are prescribed to fewer than 100 patients. Some medications are only 'Steady' or 'No', meaning we will not be able to determine what impact changing the dosage has on risk of rehospitalization. It will be important to remove some of these columns during feature selection, to ensure that our model is robust.

```
In [29]: #Counts the number of patients prescribed each medication (not status 'No')
med_counts['Prescribed'] = med_counts['Steady'] + med_counts['Up'] + med_counts['Do

#Plots the number of patients prescribed each medication, where n_patients > 100
```

```
fig = px.histogram(med_counts[med_counts['Prescribed'] > 100].reset_index(), x = 'P
                    height = 500, width = 800)
fig.update_layout(xaxis_title = '# of Patients (Log Scale)', yaxis_title = 'Medicat
fig.show()
```

